

ULTIMATE COMP ORG EXAM GUIDE (CAVEMAN EDITION)

1. MIPS DATAPATH & CONTROL

Goal: Trace how an instruction flows through hardware. * **R-Type (add, sub):** PC -> IMEM -> RegFile (read 2) -> ALU -> RegFile (write). * Control: RegDst=1, ALUSrc=0, MemtoReg=0, RegWrite=1, MemWrite=0. * **lw (Load Word):** PC -> IMEM -> RegFile (read 1) -> ALU (+ sign-extend imm) -> DMEM (read) -> RegFile (write). * Control: RegDst=0, ALUSrc=1, MemRead=1, MemtoReg=1, RegWrite=1. * **sw (Store Word):** PC -> IMEM -> RegFile (read 2) -> ALU (+ sign-extend imm) -> DMEM (write). * Control: ALUSrc=1, MemWrite=1, RegWrite=0. * **beq (Branch Equal):** PC -> IMEM -> RegFile (read 2) -> ALU (subtract). If Zero=1 -> PC = PC + 4 + (sign-extend imm * 4). * Control: Branch=1, ALUOp=01.

2. PIPELINING & HAZARDS

Goal: Do multiple things at once (IF, ID, EX, MEM, WB). **Problem:** Instructions bump into each other (Hazards).

- **Data Hazard:** Instruction needs data from a previous instruction that isn't finished.
 - Example: add \$t0, \$t1, \$t2 followed by sub \$t3, \$t0, \$t4.
 - Solution 1 (Stall): Insert NOPs (bubbles). Wastes cycles.
 - Solution 2 (Forwarding/Bypassing): Hardware wires output of EX/MEM directly back into ALU.
- **Load-Use Hazard:** You load data (lw) and immediately use it. Forwarding is not enough (data only ready after MEM).
 - Solution: MUST stall 1 cycle, then forward.
- **Control Hazard:** Branch instructions. Don't know next PC until EX stage.
 - Solution 1: Stall until branch decided (wastes 3 cycles).
 - Solution 2: Predict branch not taken. Flush pipeline if wrong.
 - Solution 3: Branch Delay Slot (put a safe instruction right after branch).

3. CACHE MEMORY (THE BIG MATH)

Goal: CPU needs data fast. Main memory slow. Cache is small & fast. Every memory address splits into 3 pieces: [TAG | INDEX | OFFSET]

Step-by-step Cache Math: 1. **Offset:** Block size. If Block = \$16\$ bytes (2^4), Offset = 4 bits. 2. **Index:** Number of sets. * Direct Mapped: 1 block per set. Number of sets = (Cache Size) / (Block Size). If 1024 sets (2^{10}), Index = 10 bits. * N-Way Set Associative: Sets = (Cache Size) / (Block Size * N). * Fully Associative: 1 giant set. Index = 0 bits. 3. **Tag:**

Leftover bits. Tag = (Address size) - Index - Offset. If 32-bit address, Tag = $32 - 10 - 4 = 18$ bits.

Hit Rate: (Hits) / (Total Accesses). Miss Penalty: Cost to fetch from Main Memory. AMAT (Average Memory Access Time): $\text{HitTime} + (\text{MissRate} * \text{MissPenalty})$.

4. VIRTUAL MEMORY

Goal: Fake infinite RAM using Disk. * **Page:** Chunk of memory (usually 4KB). * **Virtual Address:** CPU generates this. [Virtual Page Number (VPN) | Page Offset]. * **Physical Address:** Real RAM location. [Physical Page Number (PPN) | Page Offset]. * **Page Table:** Map stored in RAM. Translates VPN -> PPN. * **TLB (Translation Lookaside Buffer):** Hardware cache inside CPU just for the Page Table. Extremely fast translation. * TLB Hit: Fast translation. * TLB Miss: CPU walks page table in RAM (slow). * Page Fault: Data not in RAM, must fetch from Disk (VERY slow).

5. ASSEMBLY (MIPS/RISC-V) QUICK TRANSLATION

- `add $t0, $s1, $s2 => $t0 = $s1 + $s2`
- `lw $t0, 8($s3) => Load word into $t0 from MemoryAddress($s3 + 8).`
- `sw $t0, 0($s1) => Store word from $t0 into MemoryAddress($s1 + 0).`
- `bne $t0, $t1, Label => If $t0 != $t1, jump to Label.`
- `j Label => Jump directly to Label.`

EXAM STRATEGY (CAVEMAN MODE): 1. See Circuit? Find the red wires. Trace data. 2. See Cache Math? Find Address Size, Block Size, Cache Size. Power of 2 everything. 3. See Pipeline? Look for \$t registers sharing names. Draw arrows. Forwarding fixes math, Load-Use needs 1 bubble.